37.2 Docker 部署

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200k tokens，输出 ~6k tokens（本节）

Docker 是将 OpenClaw 部署到服务器的推荐方式，特别适合 VPS 和 NAS 场景。

37.2.1 Dockerfile 分析：多阶段构建、非 root 用户

OpenClaw 的主 Dockerfile 基于 `node:22-bookworm`：

```dockerfile
# Dockerfile
FROM node:22-bookworm

# 安装 Bun（构建脚本需要）
RUN curl -fsSL https://bun.sh/install | bash
ENV PATH="/root/.bun/bin:${PATH}"

RUN corepack enable

WORKDIR /app

# 可选的额外 apt 包（通过构建参数控制）
ARG OPENCLAW_DOCKER_APT_PACKAGES=""
RUN if [ -n "$OPENCLAW_DOCKER_APT_PACKAGES" ]; then \
      apt-get update && \
      DEBIAN_FRONTEND=noninteractive apt-get install -y \
        --no-install-recommends $OPENCLAW_DOCKER_APT_PACKAGES && \
      apt-get clean && rm -rf /var/lib/apt/lists/*; \
    fi

# 依赖安装（利用 Docker 层缓存）
COPY package.json pnpm-lock.yaml pnpm-workspace.yaml .npmrc ./
COPY ui/package.json ./ui/package.json
COPY patches ./patches
COPY scripts ./scripts
RUN pnpm install --frozen-lockfile

# 构建
COPY . .
RUN OPENCLAW_A2UI_SKIP_MISSING=1 pnpm build
ENV OPENCLAW_PREFER_PNPM=1
RUN pnpm ui:build

ENV NODE_ENV=production

# 安全加固：非 root 用户运行
RUN chown -R node:node /app
USER node

# 默认命令：启动 Gateway
CMD ["node", "openclaw.mjs", "gateway", "--allow-unconfigured"]
```

关键设计决策：

决策	原因
`node:22-bookworm` 而非 `alpine`	需要完整的 glibc 和编译工具链（Sharp 图像库等原生模块）
安装 Bun	部分构建脚本使用 Bun 运行
`OPENCLAW_PREFER_PNPM=1`	ARM/Synology 架构上 Bun 可能失败，强制用 pnpm 构建 UI
先 COPY 依赖文件再 COPY 源码	利用 Docker 层缓存，依赖不变时跳过 `pnpm install`
`USER node`	以非 root 用户（uid 1000）运行，减少容器逃逸风险
`--allow-unconfigured`	允许无配置启动，方便首次运行

> **衍生解释 — Docker 层缓存**
>
> Docker 镜像由多个只读层组成。每条 `RUN` / `COPY` 指令创建一层。如果某层的输入没有变化，Docker 会复用缓存而不重新执行。将 `package.json` 和 `pnpm-lock.yaml` 先单独 COPY 并 install，意味着只要依赖不变，即使源码改变也不需要重新安装依赖——这在开发迭代中能节省大量构建时间。

37.2.2 `docker-compose.yml` 解析

`docker-compose.yml` 定义了两个服务：

```yaml
services:
  openclaw-gateway:
    image: ${OPENCLAW_IMAGE:-openclaw:local}
    environment:
      HOME: /home/node
      TERM: xterm-256color
      OPENCLAW_GATEWAY_TOKEN: ${OPENCLAW_GATEWAY_TOKEN}
    volumes:
      - ${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw
      - ${OPENCLAW_WORKSPACE_DIR}:/home/node/.openclaw/workspace
    ports:
      - "${OPENCLAW_GATEWAY_PORT:-18789}:18789"
      - "${OPENCLAW_BRIDGE_PORT:-18790}:18790"
    init: true
    restart: unless-stopped
    command: ["node", "dist/index.js", "gateway",
              "--bind", "${OPENCLAW_GATEWAY_BIND:-lan}",
              "--port", "18789"]

  openclaw-cli:
    image: ${OPENCLAW_IMAGE:-openclaw:local}
    environment:
      HOME: /home/node
      BROWSER: echo              # 禁止自动打开浏览器
    volumes:
      - ${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw
      - ${OPENCLAW_WORKSPACE_DIR}:/home/node/.openclaw/workspace
    stdin_open: true             # 支持交互式输入
    tty: true                    # 分配终端
    init: true
    entrypoint: ["node", "dist/index.js"]
```

Gateway 服务是核心，长期运行。**CLI 服务**是按需使用的交互式容器，通过 `docker compose run openclaw-cli chat` 启动聊天。

关键配置解析：

配置	说明
`init: true`	使用 `tini` 作为 PID 1 进程，正确传递信号和回收僵尸进程
`restart: unless-stopped`	崩溃自动重启，除非手动停止

配置	说明
`--bind lan`	绑定到局域网地址（`0.0.0.0`），容器内必须如此才能从宿主访问
Volume 挂载	配置和工作区目录持久化到宿主机
`BROWSER: echo`	CLI 容器中禁止打开浏览器（容器内无 GUI）

> **衍生解释 — init: true 与 PID 1 问题**
>
> 在 Linux 容器中，`CMD` 指定的进程直接作为 PID 1 运行。PID 1 有特殊责任：回收孤儿进程、正确响应 SIGTERM 信号。Node.js 进程不是为此设计的——它不会回收子进程的退出状态，也可能不会优雅响应信号。`init: true` 会在容器中注入 `tini`（一个极小的 init 进程）作为 PID 1，它负责信号转发和僵尸进程回收。

37.2.3 Docker 沙箱（`Dockerfile.sandbox`）

OpenClaw 的代码执行沙箱可以运行在独立的 Docker 容器中：

```
# Dockerfile.sandbox
FROM debian:bookworm-slim

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
  && apt-get install -y --no-install-recommends \
    bash ca-certificates curl git jq python3 ripgrep \
  && rm -rf /var/lib/apt/lists/*

CMD ["sleep", "infinity"]
```

这个沙箱容器极其精简——只有基础 CLI 工具，没有 Node.js 或其他运行时。AI Agent 的 `exec` 工具在此容器中执行命令，与宿主系统完全隔离。

另外还有 `Dockerfile.sandbox-browser`，在沙箱基础上添加了浏览器自动化工具（Playwright/Puppeteer），用于需要浏览器能力的Agent 任务。

37.2.4 环境变量配置

Docker 部署中最重要的环境变量：

环境变量	说明	默认值
`OPENCLAW_GATEWAY_TOKEN`	Gateway 认证令牌	（无，需设置）
`OPENCLAW_GATEWAY_PASSWORD`	Gateway 密码认证	（无）
`OPENCLAW_GATEWAY_BIND`	绑定地址	`lan`
`OPENCLAW_GATEWAY_PORT`	Gateway 端口	`18789`
`OPENCLAW_BRIDGE_PORT`	Bridge 端口	`18790`
`OPENCLAW_CONFIG_DIR`	配置目录映射	（需设置）
`OPENCLAW_WORKSPACE_DIR`	工作区目录映射	（需设置）

API 密钥可以通过环境变量传递，也可以写在挂载的配置文件中。

本节小结

1. **Dockerfile 使用 node:22-bookworm**，安装 Bun + pnpm，利用层缓存优化构建，以非 root 用户运行。
2. **docker-compose 定义两个服务**：长期运行的 Gateway 和按需使用的 CLI，均使用 tini 作为 init 进程。
3. **沙箱容器**基于 debian:bookworm-slim，仅包含基础 CLI 工具，为Agent 的 exec 工具提供安全隔离环境。
4. **环境变量**控制认证、网络绑定和目录映射，是容器化部署的主要配置方式。